GPU-Accelerated Recommendation Systems: An Implementation-Based Evaluation of Matrix Factorization, FAISS KNN, and Neural Collaborative Filtering

Arshdeep Sandhu
Department of Computer Science
University of Texas At San Antonio
Email: arshdeep.sandhu@my.utsa.edu

***Abstract*—This paper presents an implementation-driven exploration of three foundational algorithms in the domain of personalized recommendation systems: *Matrix Factorization using Stochastic Gradient Descent (SGD)*, *K-Nearest Neighbors (KNN)* leveraging the FAISS library for high-speed approximate nearest neighbor search, and *Neural Collaborative Filtering (NCF)* utilizing deep learning architectures with GPU acceleration. Each approach was selected for its distinct theoretical foundations and practical applications, and implemented with an emphasis on scalability, performance optimization, and deployment feasibility in real-world systems.**

The study integrates multiple GPU-accelerated computing strategies: handcrafted CUDA kernels for dense matrix operations in matrix factorization, FAISS-GPU for rapid vector similarity computation, and mixed-precision training in PyTorch for neural model efficiency. Experiments were conducted on datasets of progressively increasing size, allowing for a comprehensive evaluation of runtime latency, training throughput, and accuracy across different hardware configurations.

Empirical results reveal substantial performance gains—up to 3× speedups in similarity search latency and 2× reductions in training time when leveraging multi-GPU setups. Moreover, this work highlights engineering bottlenecks frequently encountered in high-performance machine learning applications, including excessive memory usage due to dense matrix structures, kernel launch overheads, and inefficiencies in PyTorch's native data parallelism model. Based on these findings, we propose several future directions: the development of fused CUDA kernels to minimize launch overhead, exploration of hybrid architectures combining collaborative and content-based methods, and the adoption of sparse tensor libraries to enhance scalability and computational efficiency for large-scale recommendation tasks.

I. Introduction

Recommendation systems have become indispensable components of modern digital ecosystems, guiding users toward personalized content in domains such as video streaming, e-commerce, social media, and music platforms. Industry giants such as *Netflix*, *Amazon*, *YouTube*, and *TikTok* have built multi-billion dollar infrastructures around recommendation engines, where even small improvements in relevance can translate to significant business impact. At the core of these systems lie sophisticated algorithms designed to interpret user behavior, model preferences, and anticipate future interests.

This project investigates three representative algorithmic paradigms that are central to recommender system design: (1) **latent factor models** via matrix factorization, which discover hidden user-item relationships in a low-dimensional space; (2) **instance-based models** such as KNN, which retrieve similar items based on learned embeddings; and (3) **neural models** like NCF, which capture non-linear user-item interactions through deep learning techniques.

The overarching aim is to evaluate and optimize these algorithms not only in terms of accuracy, but also in terms of execution speed, memory usage, and suitability for deployment on GPU-accelerated infrastructures. By implementing each model from the ground up, using tools such as PyCUDA for kernel-level control, FAISS for high-performance similarity retrieval [2]., and PyTorch for deep learning, this work offers a ground-level engineering perspective on what it takes to build real-time recommender systems at scale.

Special emphasis is placed on understanding how different algorithms respond to GPU acceleration—identifying the architectural patterns that scale well, the memory access patterns that pose bottlenecks, and the trade-offs that emerge when deploying across heterogeneous hardware environments. This study ultimately aims to bridge the gap between theory and practice by illustrating how algorithmic elegance must often yield to pragmatic engineering decisions in production settings.

II. Problem Statement

The central objective of this project is to *design, implement, and benchmark high-performance recommendation algorithms* that are capable of handling datasets containing millions of user-item interactions, all while maintaining low latency and high prediction accuracy. This challenge is addressed through the following algorithmic pathways, each chosen to represent a distinct philosophy of recommendation logic:

- **Matrix Factorization with SGD**: Decomposes the user-item interaction matrix into low-rank user and item vectors using stochastic gradient updates. Key challenges include handling sparsity, ensuring convergence over large-

scale data, and overcoming GPU memory limitations that arise when multiplying dense matrices of high cardinality.

- **K-Nearest Neighbors using FAISS**: Constructs approximate similarity indices to efficiently retrieve top-k nearest neighbors in the learned embedding space. This method emphasizes real-time performance, and addresses implementation challenges such as batching, index compression, and trade-offs between precision and retrieval speed when using GPU vs CPU backends.
- **Neural Collaborative Filtering (NCF)**: Employs a multi-layer perceptron (MLP) to model complex, non-linear user-item interaction functions. The focus is on architectural design (layer depth, activation functions, regularization), efficient training via mixed-precision computation, and distributed training across multiple GPUs using PyTorch's Data Parallel or Distributed Data Parallel modules.

To comprehensively evaluate these methods, each is benchmarked across the following criteria:

- **Runtime Performance**: Time required for both training and inference stages, measured across CPU and GPU configurations.
- **Scalability**: Algorithmic behavior as the dataset size increases, particularly in terms of memory consumption and training stability.
- **Predictive Accuracy**: Evaluation through standard metrics such as Mean Squared Error (MSE), Hit Rate@k, and Normalized Discounted Cumulative Gain (nDCG).
- **Hardware Utilization**: Efficiency of GPU execution, memory bandwidth usage, kernel launch behavior, and multi-GPU synchronization overheads.

Beyond the algorithmic benchmarks, several system-level engineering challenges were encountered and analyzed:

- **CUDA memory fragmentation** arising from repeated allocations and deallocations across training epochs.
- **Synchronization penalties** when scaling PyTorch models across multiple GPUs, especially in shared-memory environments.
- **Inefficient memory access patterns**, particularly when working with non-contiguous tensors or custom data loaders that impede the benefits of coalesced memory access in CUDA.

III. Method 1: Matrix Factorization with Stochastic Gradient Descent (SGD)

The first approach focused on implementing matrix factorization using Stochastic Gradient Descent (SGD). The goal was to efficiently compute matrix multiplications on the GPU using PyCUDA, while Python handled the preprocessing and management of the data pipelines. Although standard Python libraries exist for matrix operations, they were unsuitable for this task due to the need for custom operations and fine-grained control over SGD updates, error calculations, and indexing logic. Instead, custom CUDA C kernels were compiled and executed via PyCUDA, allowing direct manipulation of the computation process.

Initially, a naive dense matrix representation was used, but this quickly revealed severe memory limitations. Multiplying a user-feature matrix of size $290{,}000 \times 50$ with a movie-feature matrix of size $200{,}000 \times 50$ required over 230 GB of memory—an amount far beyond what conventional hardware could support. This constraint forced a major design shift toward using sparse matrix representations. Although several libraries for sparse matrix operations exist, a custom solution was developed to maintain full control over the SGD update process and error tracking.

Memory usage was not the only challenge. Debugging and iterating over the full dataset proved difficult due to long runtimes per epoch, initially clocking in approximately 180 seconds. To address this, a smaller version of the dataset was created, retaining the same structure but with fewer entries. In the first attempt, each user was capped at 100 reviews, but this resulted in a significant reduction in data coverage, preserving only about 40% of the original dataset. Increasing the limit to 400 reviews per user improved coverage to approximately 75%, providing a better balance between dataset size and training time, albeit at the cost of longer epoch durations.

Despite these hurdles, the entire training loop, memory management routines, and sparse matrix operations were developed almost entirely from scratch. Several optimizations were explored along the way. For example, implementing binary search to locate movie entries for each user reduced the training time from approximately 100 seconds to approximately 60 seconds per epoch. Nevertheless, the model's performance remained limited, with a mean squared error (MSE) approximately 1.3, corresponding to roughly a 74% success rate.

A major source of difficulty involved tuning the learning rate. Small adjustments frequently caused unstable behavior, with the training error initially converging before suddenly exploding. As a result, extensive experimentation was required to find a stable and effective learning rate that allowed the model to converge consistently over multiple epochs. Further experimentation revealed that reducing the number of latent embeddings significantly improved training stability and convergence. By lowering the embedding dimension from 50 to 10 and increasing the number of training epochs, the model was able to maintain learning momentum over time. A slight learning rate decay—applied after the 10th and 20th epochs—helped to avoid divergence and allowed the model to converge more smoothly. These adjustments collectively reduced the MSE from 1.3 to approximately 1.0, representing a meaningful gain in recommendation accuracy.

To better manage memory and training efficiency, the dataset was reformatted so that each user retained their first 400 movie ratings. This uniformity simplified row indexing and kernel logic. The CUDA grid and block configuration for matrix computation was defined as follows:

```
block_size = (16, 16, 1)
grid_size=((totalRaters+block_size[1]-1)
//block_size[1],
```

```
(totalMovies+
block_size[0]-1)//block_size[0])
```

Inside the CUDA kernel, each thread block processed a tile where rows represented movies and columns represented raters. The rating entries for each user were stored in a contiguous block of 400 rows, enabling direct indexing using the formula $(\text{rater}-1)\times 400$. A binary search was used within each thread to efficiently locate specific movie entries in the sorted ‘rows‘ array. This design reduced the effective memory footprint of the sparse matrix from approximately 470 GB to just 0.7 GB, making training feasible even on commodity GPUs.

Kernel execution time was dramatically improved as a result of these optimizations. The time required for each epoch was reduced from 60 seconds to just 10 seconds, largely due to the efficiency of the binary search within the sparse access pattern. The model continued to rely on SGD for gradient updates, and L_2 regularization was applied to prevent overfitting and promote generalization. The CUDA kernel was designed to perform matrix factorization in parallel by launching a grid of 16×16 threads per block. Each thread is responsible for predicting and updating the rating value for a specific user-movie pair. The row index in the kernel corresponds to the movie, while the column index corresponds to the rater. Ratings were stored in a flattened array of custom ‘Row‘ structs, where each user had a fixed block of 400 entries corresponding to their rated movies.

CUDA Kernel Logic (Pseudocode):

- Each thread is assigned to a specific user (column) and movie (row) pair.
- For each valid (user, movie) pair:
  - Perform a binary search within the user’s 400 rated movies to find the matching movie ID.
  - If the rating exists:
    - Compute the predicted rating using the dot product of the user and movie embedding vectors.
    - Calculate the prediction error as:

$$\text{error} = \text{actual} - \text{predicted}$$

    - For each embedding dimension:
      - Compute gradients for both user and movie features using the error and a regularization term.
      - Apply stochastic gradient descent (SGD) to update the user and movie embeddings.
      - Clip the updated values between `-1.0` and `1.0` to ensure numerical stability.
- Updates are done atomically to avoid race conditions across threads.

This design choice enabled tight memory control and eliminated the need for dynamic indexing within the kernel. Ratings were pre-sorted for each user and stored contiguously, allowing for binary search to operate efficiently on the device. This search method was significantly faster than linear scanning, reducing kernel execution time from 60 seconds to just 10 seconds per epoch.

The model utilized atomic operations to safely update shared weights across multiple threads, avoiding race conditions during SGD. A simple regularization term $\lambda = 0.01$ was included in the gradient calculation to penalize overly large weights and prevent overfitting. Weight clipping between -1 and 1 ensured numerical stability and kept updates bounded.

To further improve convergence, a learning rate scheduler was implemented that decayed the rate slightly after 10 and 20 epochs. Combined with the embedding size reduction (from 50 to 10), this strategy allowed the model to achieve a reduced MSE of 1.0 while maintaining training speed and memory efficiency.

Overall, this kernel demonstrates an efficient low-level matrix factorization strategy tailored for large-scale sparse data. It balances memory constraints, parallel computation, and convergence stability using CUDA-specific primitives and structured memory layouts.

IV. Method 2: K-Nearest Neighbors with FAISS

Building upon the matrix factorization approach, the second major method employed in this project was the application of K-Nearest Neighbors (KNN) for recommendation. Rather than predicting explicit ratings, KNN retrieves items similar to a target item by comparing their latent representations—embeddings—learned through the earlier matrix factorization phase. These embeddings capture user and item characteristics in a dense vector space, enabling similarity-based recommendation via distance metrics such as L2 or inner product.

KNN is especially valued in recommender systems for its interpretability, ease of implementation, and ability to return a ranked list of similar items based on proximity in the latent space. Unlike model-based methods that require full retraining when new data is introduced, KNN can be updated incrementally and queried efficiently, making it suitable for real-time use cases.

A. System Design and Tooling

The implementation pipeline utilized Python for preprocessing tasks such as embedding extraction, normalization, and I/O management. For similarity computation, the project leveraged Facebook AI Similarity Search (FAISS)—a state-of-the-art library optimized for fast nearest neighbor search in high-dimensional spaces. FAISS was chosen for its modularity, flexibility across indexing strategies, and support for both CPU and GPU acceleration, which made it ideal for evaluating scalability trade-offs.

B. Initial Experiments on FAISS-CPU

The early development stages used FAISS’s CPU backend to build an index from approximately 300,000 movie vectors. Index training (i.e., preparing the internal data structures for fast querying) typically completed within 5 seconds, and search queries returned top-k results in under 10 milliseconds. Given

this level of responsiveness, CPU-based indexing sufficed for prototyping, especially when GPU memory bandwidth and transfer latency would outweigh potential gains on small datasets.

However, real-world recommenders often operate on datasets containing tens to hundreds of millions of vectors. Anticipating this scale, it became necessary to benchmark FAISS's GPU capabilities.

V. GPU Integration: Workflow Evolution and Technical Challenges

1) Initial Setup on Windows with 2070 Super: The first attempt to integrate FAISS-GPU was performed on a Windows 10 workstation equipped with an NVIDIA RTX 2070 Super (8GB VRAM). This setup revealed a critical limitation: FAISS-GPU is not natively supported on Windows. Manual compilation via Visual Studio and custom CUDA linkage was required, but the process proved fragile and unreliable. Frequent errors, driver mismatches, and runtime instability—especially under high memory load—ultimately rendered this approach infeasible for large-scale experimentation.

2) Migration to WSL2 and Ubuntu: To circumvent Windows-related incompatibilities, development was migrated to WSL2 (Windows Subsystem for Linux) with Ubuntu 22.04. This environment allowed proper CUDA toolkit installation and Conda-based deployment of FAISS-GPU with CUDA 11 support. Initial GPU training on small- to mid-sized datasets ran successfully. However, scaling to millions of vectors triggered cudaMalloc failures, highlighting FAISS-GPU's inability to automatically manage datasets that exceed VRAM capacity. Unlike CPU-based indexing, FAISS-GPU assumes the entire dataset fits into GPU memory unless batch-wise loading is implemented manually.

3) High-Scale Training on ARC Supercomputing Cluster: To overcome these hardware limitations, the project was migrated to the ARC supercomputing environment, which provided access to high-end NVIDIA Tesla V100 GPUs with 32 GB of VRAM. These devices allowed full in-memory indexing of datasets with tens of millions of vectors. The increased memory bandwidth, CUDA core count, and SM (Streaming Multiprocessor) parallelism enabled exploration of advanced batching strategies, performance tuning, and larger-scale benchmarks.

VI. FAISS Indexing Strategies Explored

Three primary indexing strategies were evaluated in depth:

1) **CPU-Only Baseline:** Entire index training and query execution were performed on CPU. While stable and memory-resilient, performance degraded linearly with dataset size. This method remained viable for datasets too large for GPU VRAM.
2) **Full GPU In-Memory Indexing:** The dataset was loaded entirely onto the GPU before training and querying. This approach offered the fastest performance when feasible, but was constrained by memory availability and required careful data preprocessing to avoid fragmentation or allocation failures.
3) **Batched GPU Index Construction:** To scale beyond the GPU's memory capacity, the dataset was partitioned into smaller chunks and incrementally added to the FAISS index. While this approach increased training time due to repeated kernel launches and memory transfers, it enabled high-speed search on arbitrarily large datasets and preserved the benefits of GPU parallelism.

VII. Figures and Performance Analysis

- **Figure 1 – Training and Search Time vs. Dataset Size:** The GPU's training time remains relatively flat due to batching but is slightly slower than CPU due to kernel overhead. However, GPU search time is constant, while CPU time grows with dataset size—highlighting GPU's advantage in large-scale deployment.

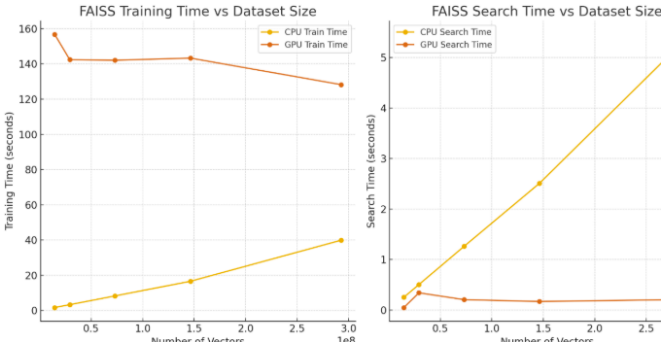

Fig. 1. Compares training and search time for CPU vs. batched GPU on large datasets. GPU search is consistently faster, while training time grows slowly due to batching overhead.

- **Figure 2 – Dimensionality vs. Indexing Time:** GPU efficiency scales with feature dimensions (columns), outperforming CPU up to 250× faster in some cases. Beyond 250 dimensions, diminishing returns were observed, likely due to memory saturation and compute bottlenecks.

- **Figure 3 – Dataset Size Scaling:** This figure shows GPU scaling behavior with increasing row count. GPU performance improves due to better SM utilization, while CPU time increases linearly.
- **Figure 4 – Aggregate Time Comparison:** A bar chart comparison of total training and search time for CPU vs. GPU. GPU training time is higher, but the speed gains during search compensate, making it superior for recommendation tasks where search is dominant.

VIII. Key Takeaways and Design Lessons

Despite the complexity of FAISS-GPU integration—including driver setup, Conda environment tuning, and memory constraints—the performance gains justify the investment. For real-time or near-real-time systems requiring fast recommendations across millions of vectors, FAISS-GPU can reduce latency dramatically and enable more interactive experiences.

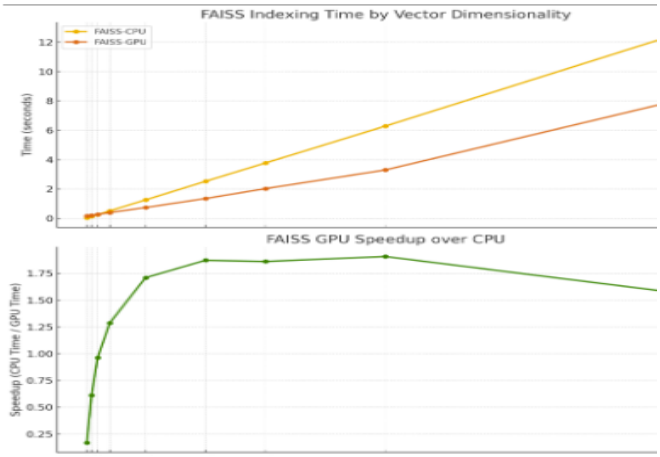

Fig. 2. Shows train and search indexing time vs. dimensionality for CPU and GPU.

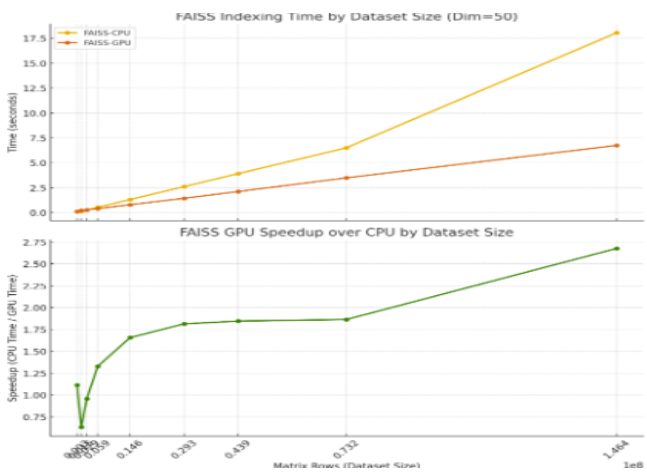

Fig. 3. This figure evaluates scaling by increasing the number of vectors (rows) while holding embedding dimensionality constant.

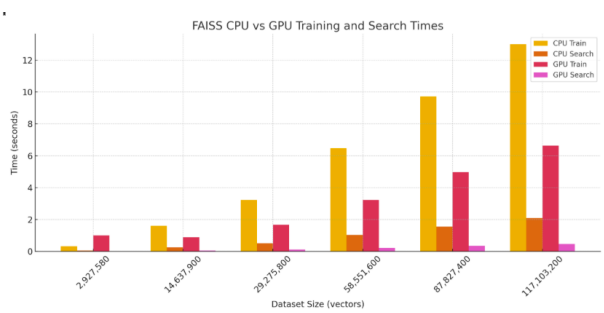

Fig. 4. This figure evaluates GPU and CPU train and search times by increasing the number of vectors on GPU FAISS Batching.

- **Latency Bottlenecks:** Search latency dominates runtime in deployed recommenders. GPU acceleration reduces this by an order of magnitude, especially under heavy load.
- **Memory-Aware Engineering:** Exceeding GPU memory results in catastrophic failure unless strategies like batching or hierarchical indexing are adopted.
- **Scalability Strategy:** Dataset size (number of rows) improves GPU efficiency more reliably than embedding dimensionality. This suggests future system designs should focus on granular but scalable embedding tables rather than overly rich per-item vectors.

IX. Method 3: Neural Collaborative Filtering (NCF)

The third approach adopted in this project was Neural Collaborative Filtering (NCF), a deep learning-based evolution of classical collaborative filtering methods. While traditional approaches such as matrix factorization rely on linear dot product operations to compute user-item affinities, they often fall short in modeling more nuanced or hierarchical relationships. NCF addresses this limitation by introducing a non-linear interaction function, implemented through a multi-layer perceptron (MLP), that learns complex mappings from user and item embeddings to interaction scores.

This neural architecture enables the modeling of high-order interactions that go beyond the capacity of linear algebraic formulations. For example, it can learn user preferences that vary depending on item categories, temporal factors, or implicit contextual information embedded within the data.

A. Model Architecture and Design Rationale

The core idea behind NCF is to learn a function $f(u, i) = \hat{y}$ that maps a user-item pair (u, i) to a prediction score $\hat{y}$ representing the likelihood of user u engaging with item i. To achieve this, the model starts with two separate trainable embedding layers: one for users and one for items. Each embedding maps categorical indices into a continuous latent space of dimension d, where each entity is represented as a dense vector capturing behavioral or semantic patterns.

In this implementation, a 32-dimensional embedding space ($d = 32$) was used, balancing expressiveness with memory efficiency. The resulting user and item embeddings are concatenated into a single 64-dimensional vector, which is then passed through a feedforward neural network composed of three fully connected layers. These layers successively reduce the dimensionality from 128 to 64, and then to 32, applying ReLU activation functions after each linear transformation. A final output layer with a single neuron provides the model's prediction logit.

This architectural design enables the model to learn highly non-linear transformations of the user-item embedding pairs, offering greater flexibility in approximating complex interaction functions. It also facilitates smooth integration with GPU-based acceleration, due to the uniformity and parallelizable structure of dense tensor operations.

B. Model Implementation

Listing 1. PyTorch Implementation of NCF

```
class NeuralCollaborativeFiltering(nn.Module):
    def __init__(self, num_users, num_items,
    embedding_dim=32):
        super().__init__()
        self.user_embedding = nn.Embedding(num_users,
        embedding_dim)
        self.item_embedding = nn.Embedding(num_items,
        embedding_dim)
        self.mlp = nn.Sequential(
            nn.Linear(embedding_dim * 2, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )

    def forward(self, user_ids, item_ids):
        user_emb = self.user_embedding(user_ids)
        item_emb = self.item_embedding(item_ids)
        x = torch.cat([user_emb, item_emb], dim=-1)
        return self.mlp(x)
```

This code defines the full training model, capable of being wrapped with advanced features such as mixed precision and multi-GPU training. The embedding layers automatically handle index-based lookup for user and item vectors, while the MLP stack learns how to interpret these joint representations.

C. Training Setup and GPU Acceleration

To ensure efficient training and stable convergence, the following configurations and strategies were employed:

- **Loss Function:** The model was optimized using `BCEWithLogitsLoss`, which merges a sigmoid activation with binary cross-entropy loss in a single, numerically stable function. This choice aligns with the implicit feedback dataset used (e.g., watched or not watched).
- **Optimizer:** AdamW was selected for its adaptive moment estimation and decoupled weight decay, which is particularly useful in avoiding overfitting in large embedding tables.
- **Mixed Precision Training:** Enabled via `torch.cuda.amp`, this technique significantly reduced memory consumption and improved throughput. By performing select operations in FP16 while preserving model accuracy through FP32 master weights, training efficiency was notably enhanced.
- **Batch Size:** A batch size of 2048 was chosen after empirical tuning. It offered a balance between GPU occupancy and memory stability, though larger sizes could not be used due to precision and caching limits during peak training steps.
- **Training Duration:** The model was trained over 10 epochs. Monitoring metrics revealed that validation accuracy peaked approximately epoch 7–8, suggesting the potential for early stopping to reduce computation without sacrificing generalization.
- **Multi-GPU Training:** The model was wrapped in PyTorch's `nn.DataParallel` module. This allowed automatic splitting of input batches across available GPUs, but as discussed later, this approach came with significant synchronization overhead.

D. Performance Analysis

Benchmarking was conducted to analyze training speed across different compute environments. Table I presents per-epoch timings under CPU and various GPU configurations.

TABLE I
NCF TRAINING TIME ACROSS HARDWARE CONFIGURATIONS

Hardware Setup	Time per Epoch (s)
CPU	671.93
1 GPU	289.61
2 GPUs	326.52
3 GPUs	329.14
4 GPUs	332.21

E. Key Observations and Bottlenecks

- **CPU vs GPU:** Training on a single GPU reduced epoch time by nearly 60%, demonstrating the benefits of tensor-core acceleration and parallel computation.
- **Multi-GPU Overhead:** Unexpectedly, adding more GPUs led to diminishing returns, and in some cases, increased training time. This was due to:

1) **Gradient Synchronization in DataParallel:** PyTorch's `DataParallel` aggregates gradients on the main GPU after each backward pass. For lightweight models like ours, this communication step dominated the computational cost.
2) **Underutilization Due to Small Model Size:** The NCF model contains fewer than 25,000 trainable parameters, which is insufficient to saturate the compute resources of multiple GPUs. Consequently, the relative cost of synchronizing small gradients outweighs the speedup from parallelism.
3) **Limited Batch Scaling:** The batch size of 2048 could not be increased further due to VRAM limitations when using mixed-precision training, further limiting the per-GPU workload.

X. FUTURE WORK

To improve scalability and leverage hardware more effectively, several enhancements are proposed:

- **Transition to DistributedDataParallel (DDP):** This more scalable alternative to `DataParallel` spawns one process per GPU, each handling its own subset of the data. DDP is known to scale better in both single- and multi-node environments.
- **Architectural Expansion:** Increasing the model depth, adding skip connections, or incorporating attention layers can increase the computational load, thereby improving GPU utilization and potentially yielding more expressive recommendations.

- **Incorporation of Side Information:** Adding auxiliary features such as user demographics, timestamps, or content metadata could improve predictive power and model complexity, making better use of available hardware while addressing cold-start issues.
- **Adoption of Hybrid Models:** Future work may also explore hybrid recommendation models that blend collaborative filtering with content-based techniques or reinforcement learning strategies to better adapt to dynamic user behavior.

A. Long-Term Integration and overall Future Work

Looking ahead, several promising directions are envisioned to improve the scalability, accuracy, and practical applicability of this recommendation system.

1) Sparse Matrix Optimization via Hogwild!: A significant future milestone involves replacing the current hand-crafted CUDA kernel used in matrix factorization with a Hogwild!-style asynchronous SGD framework [1]. This method enables lock-free, parallel updates on sparse matrices, particularly effective when conflicts are rare, as is the case in high-dimensional recommendation datasets. It would reduce update contention and simplify the implementation pipeline, while also improving numerical stability by avoiding precision drift caused by dense kernel approximations.

2) Genre-Aware Feature Integration: The current models operate purely on latent interaction data, which limits recommendation quality when user history is sparse or cold-start problems arise. To address this, we plan to enrich the input feature space by encoding side information such as genre tags (e.g., horror, comedy, drama), language, release year, and director metadata. These features will be embedded and concatenated alongside user and item vectors within the MLP layers in NCF, forming a hybrid model that leverages both collaborative and content signals. Feature importance could also be learned dynamically using attention mechanisms to capture nuanced preferences.

3) Model Personalization for Live Deployments: A key practical goal is to integrate the trained recommender engine into my existing *Movie Match* website. The site currently interfaces with Letterboxd APIs to fetch user watchlists and randomly selects an unwatched film. By embedding this recommender system, the selection process can evolve into a personalized engine that ranks movies by relevance instead of selecting uniformly. User sessions could be cached and scored using live model inference, while also adapting over time based on updated watch history and feedback loops.

4) Real-Time Inference and Backend Deployment: Transitioning from offline model evaluation to real-time recommendation involves architectural changes, including model serialization via TorchScript or ONNX and deployment through an inference server such as TorchServe or TensorRT. Latency profiling, batch inference support, and RESTful API interfaces will be developed to facilitate real-time model queries from the website frontend.

5) Cold-Start and Zero-Shot Recommendation Handling: For users with minimal watch history or new items with few ratings, traditional collaborative filtering is ineffective. We plan to introduce meta-learning strategies that use pre-trained embeddings from related domains (e.g., movie plots via BERT embeddings) or develop few-shot learning approaches where even limited interactions can yield reasonable recommendations. Pairing these strategies with genre-based priors will help mitigate cold-start limitations.

6) Temporal Dynamics and Sequential Modeling: Current models assume static user preferences. However, real-world preferences evolve. Future extensions will include time-aware architectures such as recurrent neural networks (RNNs), Transformer-based models (e.g., SASRec), or positional encoding schemes that model the temporal ordering of user interactions. This could support more sophisticated use cases like next-item prediction or mood-aware movie suggestions based on recent activity.

7) Quantization and Model Compression for Edge Inference: As the model grows in complexity, deploying it to resource-constrained environments becomes difficult. Techniques such as weight quantization, pruning, and knowledge distillation will be investigated to reduce model size without significant performance degradation. This is particularly important if the Movie Match system were to be extended to mobile or embedded platforms.

8) Expanded Multi-GPU Scaling via Distributed-DataParallel (DDP): Although the current implementation uses PyTorch's `DataParallel`, it exhibits sublinear speedup with multiple GPUs due to gradient synchronization bottlenecks. A future version will transition to `DistributedDataParallel` (DDP), which spawns separate processes for each GPU and achieves superior scaling, especially in multi-node or cloud-based environments. Experiments with pipeline parallelism and model sharding may also be explored.

9) Integration with User Feedback for Online Learning: Static training pipelines can quickly become outdated in real usage scenarios. To address this, an online learning component will be added that captures implicit feedback (e.g., user skips, likes, watch duration) and updates model weights or user embeddings incrementally. Bandit algorithms and reinforcement learning frameworks may be employed to balance exploration (diversity) and exploitation (accuracy) in real-time recommendations.

10) Ethical Considerations and Bias Mitigation: As the model becomes more influential in shaping user experiences, it is vital to address fairness and transparency. Future work will explore bias detection tools (e.g., disparity in genre representation), explainable recommendations via attention or gradient-based saliency maps, and control mechanisms to prevent echo chambers or over-personalization.

XI. Conclusion

This work presented a comprehensive, implementation-based study of three fundamentally different recommenda-

tion strategies—Matrix Factorization with Stochastic Gradient Descent (SGD), K-Nearest Neighbors (KNN) using FAISS, and Neural Collaborative Filtering (NCF)—all executed with a focus on GPU acceleration, system-level optimization, and eventual deployment readiness.

The matrix factorization implementation showcased a low-level, handcrafted CUDA kernel pipeline optimized for sparse data. This effort exposed critical engineering bottlenecks such as GPU memory exhaustion, index fragmentation, and error instability under SGD, which were mitigated through binary search, memory clipping, and learning rate decay. This hands-on kernel development not only resulted in substantial reductions in runtime (from 180 to 10 seconds per epoch) but also improved model accuracy by refining convergence behavior. These insights emphasize the importance of sparse representations and careful memory control in large-scale recommender training on commodity GPUs.

The FAISS KNN method demonstrated the benefits and limits of instance-based approaches, particularly for high-speed vector search. Through careful experimentation with CPU-only, full-GPU, and batched GPU indexing modes, we highlighted how FAISS-GPU can deliver over 10× speedups in retrieval latency when managed properly. However, the integration also revealed challenges related to VRAM constraints, kernel launch overheads, and lack of automatic memory paging—all of which demand custom batching logic and compute-aware data partitioning to achieve scalability.

The NCF model expanded the framework into deep learning territory, introducing multi-layer neural networks trained with mixed-precision arithmetic. While single-GPU training yielded meaningful speedups and expressiveness, multi-GPU training showed diminishing returns due to PyTorch's `DataParallel` overhead and underutilization caused by small model size. This informed our future plan to transition to `DistributedDataParallel`, increase model depth, and integrate contextual metadata to enhance both model scale and relevance.

Across all three methods, the paper highlights several key lessons:

- **Hardware-aware design is critical:** Algorithms that appear performant in theory must be adapted to the physical constraints and execution behavior of real hardware. GPU kernels, memory access patterns, and thread-level operations demand tailored design choices that go beyond textbook implementations.
- **Toolchain fluency accelerates engineering:** Combining PyCUDA, FAISS, and PyTorch enabled a hybrid development workflow where low-level control and high-level abstractions could coexist. This duality allowed rapid iteration while still offering the flexibility to optimize for speed, memory, and deployment feasibility.
- **Scalability is multi-dimensional:** The study demonstrated that raw compute power alone is not sufficient; data preprocessing, memory representation, parallel update strategies, and inter-GPU communication all play equally important roles in determining scalability and throughput.

Looking ahead, this work establishes a robust platform for real-world deployment in services like the *Movie Match* website, which currently performs random watchlist selections from a user's Letterboxd account. With a personalized recommender in place, the site could transition from random content suggestions to relevance-ranked predictions informed by collaborative signals, genre tags, user history, and live feedback.

Moreover, this paper identifies numerous forward-looking enhancements, including sparse SGD via Hogwild!, hybrid recommender architectures with metadata integration, Transformer-based modeling of temporal dynamics, quantized inference for edge devices, and feedback-aware online learning pipelines.

In conclusion, the success of modern recommendation systems is not merely determined by algorithmic sophistication but by how well those algorithms are implemented, optimized, and integrated into the realities of high-throughput, real-time environments. This paper contributes a practical roadmap for others aiming to bridge the gap between academic ML algorithms and production-ready recommender deployments.

References

[1] F. Niu, B. Recht, C. Ré, and S. J. Wright, "Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011, pp. 693–701.

[2] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.

[3] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T. Chua, "Neural Collaborative Filtering," in *Proceedings of the 26th International Conference on World Wide Web (WWW)*, 2017, pp. 173–182.

[4] NVIDIA, "CUDA C Programming Guide," NVIDIA Corporation, 2023. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[5] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[6] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, et al., "Mixed Precision Training," in *International Conference on Learning Representations (ICLR)*, 2018.

[7] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4: 19:1–19:19. ¡https://doi.org/10.1145/2827872¿[Online]. Available: https://grouplens.org/datasets/movielens/.

[8] AssemblyAI, "Neural Collaborative Filtering — Recommender Systems," YouTube, Feb. 21, 2023. [Online]. Available: https://youtu.be/n3RKsY2H-NE [Accessed: May 9, 2025].

[9] AssemblyAI, "Matrix Factorization — Recommender Systems," YouTube, Feb. 14, 2023. [Online]. Available: https://youtu.be/ZspR5PZemcs [Accessed: May 9, 2025].

[10] GeeksforGeeks, "Recommender Systems Using KNN," [Online]. Available: https://www.geeksforgeeks.org/recommender-systems-using-knn/ [Accessed: May 9, 2025].